Your first SIMD kernel in C++

A practical guide for Project 1

Write the implementation yourself, one small step at a time. Your kernel files now contain paired scalar/SIMD declarations and throwing TODO bodies; main.cpp remains empty. Nothing currently computes, checks, or times a kernel. A successful run today only confirms that the project builds.

This guide uses SAXPY as the first exercise: update each output value by multiplying the corresponding input by one coefficient and adding the old output. The mathematical rule is y[i] = a * x[i] + y[i]. Start with float; add double later. The current scaffold selects explicit intrinsics; this tutorial also explains auto-vectorization, which the assignment separately requires.

1. Understand what the compiler can do

SIMD means Single Instruction, Multiple Data. A scalar arithmetic instruction handles one value; a packed SIMD arithmetic instruction handles several values in lanes. For example, a 256-bit register can hold eight 32-bit floats or four 64-bit doubles. Register capacity is not a promise of that much speedup.

A numeric kernel is the small function that does the repeated numerical work. It should not allocate memory, print results, or choose experiment settings. Keep those tasks in the calling program so correctness and timing remain understandable.

Your first route is ordinary C++ plus compiler auto-vectorization: write independent operations over contiguous elements and let the compiler choose instructions. GCC and Clang analyze safety and cost; a loop may remain scalar, use runtime checks, or have both vector and scalar paths. [1, 4]

Explicit intrinsics are another route. They expose architecture-specific operations and vector types, often through headers such as immintrin.h on x86. You then take responsibility for instruction support, loads/stores, and tails. They are unnecessary for your first working kernel and are not a guarantee of better performance.

std::vector<float> is a dynamic array container. Its name does not mean that its operations automatically use SIMD. It is useful here because its elements occupy contiguous storage and it manages their lifetime.

Your route through the files

- src/simd/saxpy.hpp: choose declarations and document the contract.
- src/simd/reference.cpp: implement a clear scalar reference.
- src/simd/saxpy.cpp: write the scalar AXPY baseline; saxpy_simd.cpp: write its explicit SIMD partner.
- src/simd/main.cpp: initialize inputs, validate outputs, then later run experiments.

Milestone: explain in your own words what one output element should become before writing any loop.

2. Choose a safe interface

Begin in src/simd/saxpy.hpp. The scaffold now supplies guards, includes, and declarations for saxpy_scalar/saxpy_simd and daxpy_scalar/daxpy_simd. Read their contracts before replacing TODO bodies. A declaration tells other files how to call a function; its definition provides the body in a .cpp file.

Illustrative declaration only - choose your own naming consistently:

```
#include <cstddef>
void saxpy(float a, const float* x, float* y, std::size_t n);
```

float a passes a coefficient by value. const float* x points to readable float elements; this function may not change those elements through x. float* y permits changes to output elements. std::size_t is an unsigned type suitable for element counts. The pointer does not carry the array length: n is part of the contract.

Include your header in both implementation files and in main.cpp when you begin using these declarations. Give the reference and candidate different names so the caller can run each independently. Keep declarations and definitions identical in parameter types and order.

State the contract before implementing it

- x and y each provide at least n valid float elements for the duration of the call.
- y is both input and output: every value to be read must already be initialized.
- The two used ranges do not overlap. Keep separate arrays initially.
- n can be zero or any positive count; do not require a multiple of a vector width.
- Do not promise special alignment initially. Ordinary valid float alignment is enough for your interface.
- Only the first n output elements may change; input elements must remain unchanged.

For a zero-length call, arrange the implementation so it performs no element access. In the first harness, you may pass pointers into a small valid allocation even for this case. Do not invent a requirement to read the first element when n is zero.

Why non-overlap matters

If y begins one element into x's storage, writing an output can change a later input. That creates a dependence between iterations and changes what reordering is legal. const does not prove non-overlap: it restricts access through one pointer, not every possible alias.

The compiler may insert runtime overlap checks and choose a scalar fallback. [4] Start with separate storage rather than compiler-specific restrict annotations. An incorrect no-alias promise can make optimized code wrong. Standard C++17 has no standard restrict keyword.

Milestone: write a short header comment describing the allowed arrays, lengths, and side effects, then review the supplied declarations.

3. Prepare inputs and write your loop

In main.cpp, use separate containers for the input and initial output. Keep the initial output so that reference and candidate can start from identical values. Allocate once, initialize every element you will read, and keep the containers alive while pointers into them are used.

Illustrative container setup only - this is not the kernel:

```
#include <vector>
std::vector<float> input(19, 0.5f);
std::vector<float> initial_output(19, -1.0f);
// .data() supplies a pointer; .size() supplies an element count.
```

Constant values are a useful first sanity check. For a = 2, the example data should produce zero in each output position. Follow this with nonconstant positive, negative, and zero values so that a function that always writes zero will fail.

Write the reference, then the candidate

In reference.cpp, express the per-element mathematical rule directly in a simple scalar traversal. In saxpy.cpp, write your clear scalar baseline. In saxpy_simd.cpp, write its explicit intrinsic counterpart when ready. Later add a distinct ordinary-loop auto-vectorized variant to satisfy that separate assignment requirement.

Keep allocation, I/O, random-number generation, and validation outside both functions. Each output depends on its matching input and old output, rather than the previous iteration's computed result. That independence makes this a useful first vectorization exercise.

Bounds and tails

The last valid element has index n - 1 when n is positive. For n = 0, subtracting 1 from an unsigned count wraps around; do not use that expression to design an empty-input stopping condition. Never access x[n] or y[n].

Do not round the logical length upward to a lane multiple and touch extra elements. With ordinary loops, let the compiler handle leftover elements using the code it selects. If you later use intrinsics, you must explicitly arrange safe remainder processing or valid masked accesses.

A shifted starting address is not necessarily vector-aligned. A future shifted test must allocate enough extra storage for all n elements after the shift. Do not use aligned-load intrinsics or alignment assumptions unless the actual pointer satisfies their requirements.

First build

```
make CXX=g++ MODE=avx
./build/g++/avx/simd_starter
```

No output is expected until you write output or checks in main.cpp. Fix compilation errors before adding optimization features. Build success alone says nothing about numerical correctness.

4. Establish correctness before speed

In main.cpp, make two independent copies of the same initial output. Call the reference on one and the candidate on the other, using the same coefficient, unchanged input, and length. Compare every logical output element. Report the first failing index and both values, and return a nonzero exit code on failure.

Also check a few hand-calculated answers. Reference and candidate can share the same bug; agreement between them is necessary but not enough. A higher-precision calculation can help investigate float discrepancies, but it has its own rounding behavior and is not automatically a bit-exact oracle for separate float operations.

Compare floating-point values deliberately

Exact equality is too restrictive for many future variants. A common finite-value acceptance rule is absolute_error <= absolute_tolerance + relative_tolerance * abs(reference). The absolute term helps near zero; the relative term scales with magnitude.

Illustrative comparison only - choose tolerances for your data:

```
// Include <cmath>; got and expected are finite values here.
bool close = std::abs(got - expected)
           <= atol + rtol * std::abs(expected);
```

For small, bounded float inputs in this non-reduction exercise, atol = 1e-6 and rtol = 1e-5 are reasonable starting values to investigate, not universal correctness limits. Document your choice, inspect maximum error, and tighten it where justified. Reject unexpected NaNs or infinities explicitly; decide separately whether you intend to test those inputs.

Your Makefile uses -ffp-contract=off, disabling contraction into fused multiply-add. This keeps the initial comparison policy simpler. FMA rounds a combined multiply/add differently from two separately rounded operations. Do not add fast-math just to make a report say vectorized. [1, 5]

Build your own small test matrix

- Sizes: 0, 1, 2, 7, 8, 9, 15, 16, 17, 19, and a larger count.
- Values: zero, positive, negative, and deterministic mixed values; several coefficients.
- Memory: separate arrays, then valid shifted starts with enough remaining capacity.
- Integrity: input unchanged, output guards before/after the used range unchanged.

Guards can expose writes beyond the intended range. They do not reliably detect invalid reads. For additional debugging, rebuild into a separate directory with sanitizers (the scalar disabling flags remain attached to scalar files):

```
make CXX=g++ MODE=avx BUILD_DIR=build/debug \
  CXXFLAGS='-O1 -g -fsanitize=address,undefined' \
  LDFLAGS='-fsanitize=address,undefined'
./build/debug/simd_starter
```

Do not time the sanitizer build. Milestone: all your scalar/SIMD checks pass in each build mode before measuring performance.

5. Build scalar and explicit SIMD together

The scaffold now puts scalar and explicit SIMD functions in separate files sharing each kernel header. Each executable includes both. The *_scalar functions use ordinary C++; *_simd functions are for your intrinsics. All bodies currently throw until implemented, including for empty inputs.

```
make CXX=g++ MODE=avx
./build/g++/avx/simd_starter
make CXX=clang++ MODE=avx
./build/clang++/avx/simd_starter
make CXX=g++ MODE=native
./build/g++/native/simd_starter
```

Same ISA; different vectorization policy

The default MODE=avx adds -mavx; MODE=native adds -march=native. Both scalar and SIMD objects use that same target, -O3, and -ffp-contract=off. Binaries need compatible CPU/OS support and have no runtime dispatch. Native does not force the widest vectors or promise better runtime. [3]

saxpy.cpp, dot_product.cpp, elementwise_multiply.cpp, and reference.cpp additionally disable auto-vectorization: GCC uses -fno-tree-vectorize -fno-tree-slp-vectorize; Clang uses -fno-vectorize -fno-slp-vectorize. The *_simd.cpp files include immintrin.h and do not receive those disabling flags. [1, 4]

No-vectorization flags do not disable explicit intrinsics. Keep them out of scalar files. Scalar arithmetic may still use XMM registers; register names alone do not prove packed vector arithmetic. The Makefile does not enable LTO; keep it that way for this straightforward separation.

Auto-vectorized comparison is still required

The old MODE=scalar and MODE=auto switches no longer exist. Scalar and SIMD are now separate named functions, not exclusive build modes. Explicit intrinsics do not satisfy the assignment's separate auto-vectorization comparison.

Later add ordinary-loop *_auto functions in separate source files, add those files to the SIMD_SOURCES list (which compiles without scalar disabling flags), and avoid intrinsics in those functions. Verify that the compiler vectorizes them using reports and disassembly. No *_auto implementation is currently supplied.

Your file pairs

AXPY uses saxpy.hpp with saxpy.cpp and saxpy_simd.cpp. Dot product and elementwise multiply follow the same pattern. Shared headers document the scalar and SIMD contracts. main.cpp will select calls once you write the harness.

Use make -B when changing flags, or choose a new BUILD_DIR. Record the compiler version and exact commands. make clean removes build/. The README is the current build reference; this guide's ordinary-loop discussion is background for your later auto-vectorized comparison.

6. Verify that the kernel uses SIMD

For a future ordinary-loop *_auto variant, ask the compiler what happened. Explicit intrinsics need disassembly checks; they do not rely on the loop vectorizer. Replace source/object names below with the future variant when studying auto-vectorization. These commands force recompilation and capture messages for inspection. The directory creation matters because shell redirection happens before Make runs.

```
mkdir -p build/reports
make -B CXX=g++ MODE=avx \
  CXXFLAGS='-fopt-info-vec-all' \
  > build/reports/gcc-avx.txt 2>&1
```

GCC's optimization report identifies vectorized and missed loops. For your future auto-vectorized variant, check its own file and line numbers, not the scalar saxpy.cpp or a success message elsewhere in the build. [2]

```
make -B CXX=clang++ MODE=native \
  CXXFLAGS='-Rpass=loop-vectorize -Rpass-missed=loop-vectorize -Rpass-analysis=loop-vectorize' \
  > build/reports/clang-native.txt 2>&1
```

Clang's success, missed, and analysis remarks help explain decisions. [4, 5] A report may mention loop versions or remainder code. A vector path and scalar fallback can coexist. Small lengths may execute the scalar path even when a vector path exists.

Read a small portion of disassembly

```
objdump -d -C build/g++/avx/saxpy.o
objdump -d -C build/g++/avx/saxpy_simd.o
```

On an x86 machine, locate your function name and its loop. Packed float arithmetic such as mulps/addps or vmulps/vaddps is evidence of SIMD arithmetic. Scalar mulss/addss or vmulss/vaddss handles one float even though register names may look similar. YMM registers hold 256 bits; ZMM registers hold 512 bits. XMM registers can contain packed or scalar operations.

These are x86 examples, not universal mnemonics. On an ARM machine you will see different instructions. Look at arithmetic inside the repeated kernel body; a vector load or a vectorized initialization elsewhere is insufficient evidence.

If it did not vectorize

First check that you rebuilt the intended mode and are inspecting the intended object: scalar saxpy.o, explicit saxpy_simd.o, or your future auto-vectorized object. Read the missed-loop reason. Simplify the kernel, check for real cross-iteration dependencies, and consider runtime overlap checks. Do not assert alignment or independence unless your contract and callers make the assertion true.

Keep a concise evidence excerpt with the compiler version, flags, source location, observed vector width, and any scalar remainder/fallback. This supports a claim about generated code, not a speedup claim. Your actual input must reach the relevant path, and performance still needs measurement.

Milestone: explain why the scalar and explicit SIMD objects differ, or honestly explain why they do not.

7. Measure only after it works

Keep the required experiment single-threaded. Begin with a hypothesis: for example, vectorization may help cache-resident SAXPY more than a very large streaming working set. Treat this as a prediction to test, not a conclusion.

Use std::chrono::steady_clock for elapsed durations. Allocate and initialize outside the measured region. Warm up deliberately, repeat enough work for stable timings, and perform several independent trials. Check or consume outputs after timing so the work remains observable. Avoid volatile arrays as an optimization barrier; they change the memory operations you are studying.

SAXPY changes y. Decide how every trial gets its starting output back. Reset outside the timed region and document that reset can warm caches. Repeatedly updating the same output changes the experiment and can eventually overflow. Keep the same policy for scalar and vector builds.

Print or save raw per-trial durations, N, mode, compiler/flags, and relevant environment settings. Record CPU/ISA, OS, frequency policy, SMT and pinning. Report variability and alternate run order where useful to reduce drift. Do not infer cycles from wall time and an advertised CPU frequency; collect suitable cycle measurements if reporting cycles per element.

Use clear units and a fair comparison

Speedup = scalar elapsed time / vector elapsed time for the same workload. SAXPY conventionally counts two floating-point operations per element. GFLOP/s = 2*N / seconds / 1e9 for one pass. Its logical data traffic is two reads and one write: 12*N bytes for float, 24*N for double. Logical GiB/s divides bytes/seconds by 2^30.

Logical traffic is not measured DRAM traffic: caching, write behavior, and hardware details affect actual transfers. Label estimates clearly. When batching passes, include the pass count in the work calculation.

A useful negative case

Sweep N from cache-resident arrays to a footprint larger than the last-level cache. Two float arrays occupy about 8*N bytes, before extra copies and harness data. If scalar and SIMD streaming runs approach a similar bandwidth limit, speedup may compress despite clear vector instructions. This is a plausible outcome, not guaranteed: establish it with repeated data and memory-bandwidth evidence.

The logical float arithmetic intensity is roughly 2/12 FLOP per byte. This motivates a memory-bound hypothesis, but a roofline argument still requires measured bandwidth and an explained compute ceiling. One tiny timing or one vectorization message cannot establish the bottleneck.

Project 1 extends beyond this first kernel

The assignment requires at least three selected kernels, type comparisons, locality, alignment/tail and stride studies, vectorization evidence, throughput/speedup plots, roofline interpretation, and a negative case. It also asks for reproducible scripts/data, methodology, a report, and an AI-use note. This guide implements none of those experiments and changes none of your source files. [A]

References and next session

Your next small steps

1. Choose the function contract and declarations in saxpy.hpp.
2. Write the reference and candidate yourself, keeping each function focused.
3. Add deterministic inputs and independent correctness checks in main.cpp.
4. Run both scalar/SIMD functions in avx/native builds; add an auto variant later.
5. Save kernel-specific vectorization evidence, then design a timed experiment.

You are ready to move on when you can explain the indexing, the pointer lifetime, why iterations are independent, how tails stay in bounds, and what your tolerance accepts.

Official compiler documentation

These links support the compiler-specific advice. Documentation can evolve; record the version of the compiler you actually use. Checked October 18, 2026. The local build environment previously reported GCC 13.3 and Clang 18.1; recheck before collecting results.

[1] GCC 13.4: Optimize Options. Optimization levels, tree loop/SLP vectorization controls, and floating-point contraction. https://gcc.gnu.org/onlinedocs/gcc-13.4.0/gcc/Optimize-Options.html

[2] GCC 13.4: Developer Options. Optimization information through -fopt-info, including vectorization diagnostics. https://gcc.gnu.org/onlinedocs/gcc-13.4.0/gcc/Developer-Options.html

[3] GCC 13.4: x86 Options. Meaning and portability implications of -march=native on x86. https://gcc.gnu.org/onlinedocs/gcc-13.4.0/gcc/x86-Options.html

[4] LLVM: Auto-Vectorization in LLVM. Loop vectorization, runtime pointer checks, remainder handling, profitability, and diagnostic remarks. https://llvm.org/docs/Vectorizers.html

[5] Clang Compiler User's Manual. Optimization remarks and floating-point controls. https://clang.llvm.org/docs/UsersManual.html

Assignment and scope

[A] ECSE4320_6320_Class-wide_Projects (2).pdf, local assignment specification, Project 1: SIMD Advantage Profiling and common requirements. The file is in the project root. The assignment is the authority for submission requirements.

This document is instructional support, not an implementation or a completed report. No SIMD instruction set is assumed available on every CPU. The Makefile's native mode supplies a CPU-targeting option; it does not certify that the assignment's widest-available-vector study has been completed.

AI-use note: Codex wrote this guide using the local assignment and scaffold plus official compiler documentation. You remain responsible for understanding and validating your implementation, measurements, and claims.